

Design & Analysis of Algorithm (Assignment 1)

Name :- Sana Munir Alam

Class :- BCS-5E

Roll # :- 24K-0573

Q12.ii) Iteration 1:

$j=1$ $i=0$ $key = A[j] = A[1] = 7$

initial Array: $[18, 7, 25, 14, 3, 21, 10, 30]$

compare $A[i] > key \Rightarrow 18 > 7$ true

} shift 18 to right, $i--$, loop break
insert 7 at $A[i+1] = key$

current Array: $[7, 18, 25, 14, 3, 21, 10, 30]$

Iteration 2:

$j=2$, $i=1$, $key = A[j] = A[2] = 25$

compare $A[i] > key \Rightarrow 18 > 25$ false } no shifting required

current Array: $[7, 18, 25, 14, 3, 21, 10, 30]$

Iteration 3:

$j=3$, $i=2$ $key = A[j] = A[3] = 14$

compare $A[i] > key \Rightarrow 25 > 14$ true } shift 25 to right, $i--$

array: $[7, 18, 25, 25, 3, 21, 10, 30]$

$i=1$ so compare $A[i] > key \Rightarrow 18 > 14$ true } shift 18 to right, $i--$

array: $[7, 18, 18, 25, 3, 21, 10, 30]$

$i=0$ so compare $A[i] > key \Rightarrow 7 > 14$ false } loop break insert 14 at $A[i+1] = A[1] = 14$

current Array: $[7, 14, 18, 25, 3, 21, 10, 30]$

Iteration 4:

$j=4$, $i=3$, $key = A[j] = A[4] = 3$

compare $A[i] > key \Rightarrow 25 > 3$ true } shift 25 to right, $i--$

array: $[7, 14, 18, 25, 25, 21, 10, 30]$

$i=2$ so compare $A[i] > key \Rightarrow 18 > 3$ true } shift 18 to right, $i--$

array: $[7, 14, 18, 18, 25, 21, 10, 30]$

Data at start

- for $j=1$ to $n-1$
- ~~$key = A[j]$~~ $key = A[j]$
- $i = j-1$
- while $i > 0$ & $A[i] > key$

$i=1$ so compare $A[i] > \text{key} \Rightarrow 14 > 3$ true } shift 14 to right, $i--$
array: $[7, 14, 14, 18, 25, 21, 10, 30]$

$i=0$ so compare $A[i] > \text{key} \Rightarrow 7 > 3$ true } shift 7 to right, $i--$
array: $[7, 7, 14, 18, 25, 21, 10, 30]$

$i=-1$ loops break, insert 3 at $A[i+1] = A[0+1] = A[1] = 3$
current array: $[3, 7, 14, 18, 25, 21, 10, 30]$

Iteration 5:

$j=5$, $i=4$ key = $A[j] = A[5] = 21$

compare $A[i] > \text{key} \Rightarrow 25 > 21$ true } shift 25 to right, $i--$
array: $[3, 7, 14, 18, 25, 25, 10, 30]$

$i=3$ compare $A[i] > \text{key} \Rightarrow 18 > 21$ false } loop breaks, insert 21
at $A[i+1] = A[4] = 21$
current array: $[3, 7, 14, 18, 21, 25, 10, 30]$

Iteration 6:

$j=6$, $i=5$ key = $A[j] = A[6] = 10$

compare $A[i] > \text{key} \Rightarrow 25 > 10$ true } shift 25 to right, $i--$
array: $[3, 7, 14, 18, 21, 25, 25, 30]$

$i=4$ so compare $A[i] > \text{key} \Rightarrow 21 > 10$ true } shift 21 to right, $i--$
array: $[3, 7, 14, 18, 21, 21, 25, 30]$

$i=3$ so compare $A[i] > \text{key} \Rightarrow 18 > 10$ true } shift 18 to right, $i--$
array: $[3, 7, 14, 18, 18, 21, 25, 30]$

$i=2$ so compare $A[i] > \text{key} \Rightarrow 14 > 10$ true } shift 14 to right, $i--$
array: $[3, 7, 14, 14, 18, 21, 25, 30]$

$i=1$ so compare $A[i] > \text{key} \Rightarrow 7 > 10$ false } loop breaks, insert 10
at $A[i+1] = A[2] = 10$
current array: $[3, 7, 10, 14, 18, 21, 25, 30]$

Iteration 7:

$j=7$, $i=6$ key = $A[j] = A[7] = 30$

compare $A[i] > \text{key} \Rightarrow 25 > 30$ false } no shifting required
loop breaks, insert 30
at $A[i+1] = A[7] = 30$
current array: $[3, 7, 10, 14, 18, 21, 25, 30]$

Final Sorted Array : ~~[3, 7, 10, 14, 18, 21, 25, 30]~~

[3, 7, 10, 14, 18, 21, 25, 30]

1 iii) Why worst case $O(n^2)$?

The inner while loop for a given j runs once per element in $A[0, \dots, j-1]$ that is bigger than key (at most j times).

In the worst case (array in strictly decreasing order), every key is smaller than everything before it, so the inner loop runs the maximum possible j times for every j .

$$T(n) = \sum_{j=1}^{n-1} j = 1 + 2 + \dots + (n-1) = \frac{n(n-1)}{2} = \frac{n^2 - n}{2}$$

Dropping the constant factor and lower order term $T(n) = O(n^2)$.
hence $O(n^2)$.

1 iv) Loop invariant proof of correctness.

Invariant: - At the start of each iteration j (the outer for loop), the subarray $A[0, \dots, j-1]$ consists of the element originally in those positions, but in sorted order.

Initialization: -

Before the first iteration, $j=1$, so $A[0, \dots, j-1] = A[0..0]$ is a single element - trivially sorted.

Maintainance:

During each iteration, the value at $A[j]$ is stored in key. The while loop shifts every element (in the sorted position) that are greater than key, one slot to the right. Then insert key into the resulting gap. Since key is placed exactly where it belongs relative to the (already sorted) elements around it, $A[0..j]$ is sorted at the end of the iteration.

The invariant holds for $j+1$

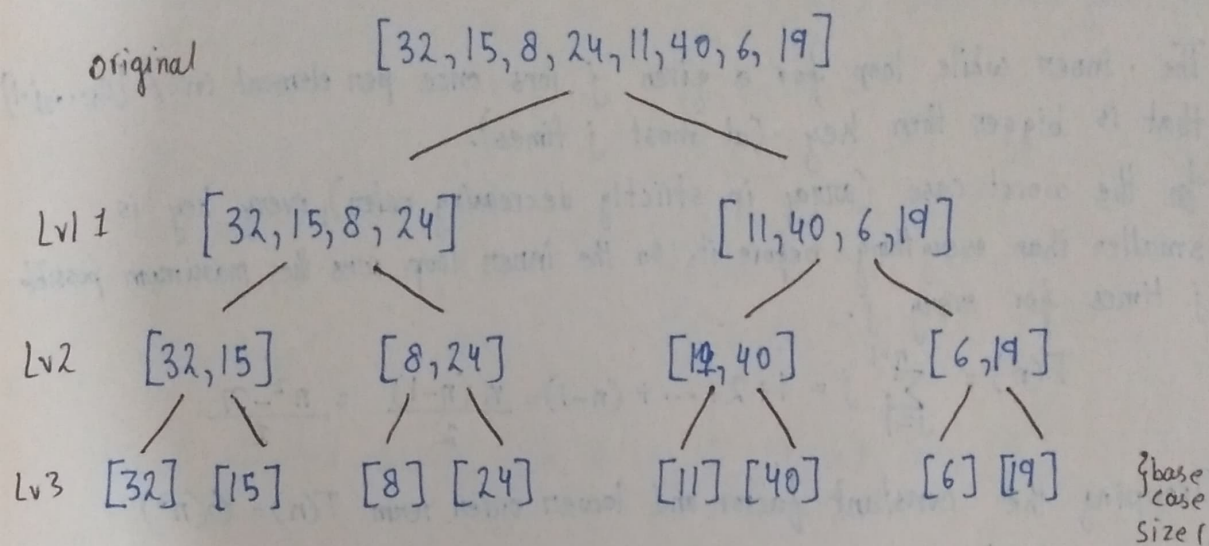
Termination

The loop ends when $j=n$, so the invariant tells us $A[0..n-1]$. The whole array is sorted. Since insertion sort only moves elements (never deletes/duplicates), these are exactly the original elements, just rearranged.

[Correctness proved]

Q2i) Array: [32, 15, 8, 24, 11, 40, 6, 19]

Step 1: Divide the array (until base case reached, size 1), split in half each time



Step 2: Sort and merge (bottom-up merges)

Merging pair of size 1 arrays.

- [32], [15] → compare 32 vs 15 → [15, 32]
- [8], [24] → compare 8 vs 24 → [8, 24]
- [11], [40] → compare 11 vs 40 → [11, 40]
- [6], [19] → compare 6 vs 19 → [6, 19]

Merging pair of size 2 array now

- [15, 32] merged with [8, 24]

compare	smaller	output so far
15 vs 8	8	[8]
15 vs 24	15	[8, 15]
32 vs 24	24	[8, 15, 24]
32 leftover	32	[8, 15, 24, 32]

- [11, 40] merged with [6, 19]

compare	smaller	output so far
11 vs 6	6	[6]
11 vs 19	11	[6, 11]
40 vs 19	19	[6, 11, 19]
40 leftover	40	[6, 11, 19, 40]

Final merge: The 2 size 4 sorted halves $[8, 15, 24, 32]$, $[6, 11, 19, 40]$

compare	winner	output so far
8 vs 6	6	$[6]$
8 vs 11	8	$[6, 8]$
15 vs 19	11	$[6, 8, 11]$
15 vs 19	15	$[6, 8, 11, 15]$
24 vs 19	19	$[6, 8, 11, 15, 19]$
24 vs 40	24	$[6, 8, 11, 15, 19, 24]$
32 vs 40	32	$[6, 8, 11, 15, 19, 24, 32]$
40 leftover	40	$[6, 8, 11, 15, 19, 24, 32, 40]$

Final Sorted Array: $[6, 8, 11, 15, 19, 24, 32, 40]$

2ii) Three way merge sort

Setup -- Divide into three parts of size $\frac{n}{3}$ each, recursively sort each, then combine with a 3-way merge.

Cost of a 3 way merge: At every output position we compare the current fronts of the 3 lists (at most 2 comparisons to find the minimum of 3 numbers, a constant amount of work), then emit one element.

Since there are "n" elements total to emit, the merge step costs $O(n)$

Recurrence:

$$T(n) = 3T\left(\frac{n}{3}\right) + O(n)$$

3 cause we recursively sort 3 parts

As original n elements divided into 3 approximately equal parts

merge part
(n, as we need to output all n elements)
Big theta ignores constant factor

apply Master Theorem

$$a = 3 \quad b = 3 \quad d = 1$$

$$a \geq b^d$$

$$3 = 3^1$$

$$\rightarrow \text{Big } O \Rightarrow O(n \log_3 n)$$

$$\rightarrow \text{Big } \Theta \Rightarrow O(n \log n)$$

$$\therefore T(n) = O(n^1 \log n) = O(n \log n)$$

Justification: The asymptotic order doesn't change. It's still $O(n \log n)$, exactly like standard 2 way merge sort.

What changes is only the base of the logarithm (depth becomes $\log_3 n$, instead of $\log_2 n$, meaning fewer recursion levels). But each level's combine step is proportionally more expensive.

These two eff cancel out in Big Θ terms since $\log_3 n$ and $\log_2 n$ differ only by the constant factor $\frac{1}{\log_2 3}$, and Θ notation ignore constant factor.

∴ The generalizes and k way merge sort with

$T(n) = kT(\frac{n}{k}) + O(n)$ gives $a=b=k$ so $a=b'$ always holds regardless of k .

Q3i) Original Array: [24, 9, 17, 5, 31, 12, 27, 3]

Data
pivot = first element

Partition 1:

low = 0 high = 7 pivot = $A[0] = 24$

$j = 1$ compare $A[j] < \text{pivot} \rightarrow 9 < 24$ true $\{ i++$

so $i = 1$ swap $A[i], A[j] \rightarrow A[1], A[1]$, $j++$

$j = 2$ compare $A[j] < \text{pivot} \rightarrow 17 < 24$ true $\{ i++$

so $i = 2$ swap $A[i], A[j] \rightarrow A[2], A[2]$, $j++$

$j = 3$ compare $A[j] < \text{pivot} \rightarrow 5 < 24$ true $\{ i++$

so $i = 3$ swap $A[i], A[j] \rightarrow A[3], A[3]$, $j++$

$j = 4$ compare $A[j] < \text{pivot} \rightarrow 31 < 24$ false $\{ j++$

no increment in i , only j increase, no swap

$j = 5$ compare $A[j] < \text{pivot} \rightarrow 12 < 24$ true $\{ i++$

so $i = 4$ swap $A[i], A[j] \rightarrow A[4], A[5]$

array: [24, 9, 17, 5, 12, 31, 27, 3]

$j = 6$ compare $A[j] < \text{pivot} \rightarrow 27 < 24$ false $\{ j++$

no increment in i , only j increase, no swap

$j = 7$ compare $A[j] < \text{pivot} \rightarrow 3 < 24$ true $\{ i++$

so $i = 5$ swap $A[i], A[j] \rightarrow A[5], A[7]$

array: [24, 9, 17, 5, 12, 3, 27, 31]

Loop ends swap $A[\text{low}], A[i] \rightarrow A[0], A[5]$

current Array: [3, 9, 17, 5, 12, 24, 27, 31]

After partition 1, pivot 24 lands at index 5. Left subarray is elements (< 24), indices 0-4 [3, 9, 17, 5, 12],

Right subarray is elements (> 24), index 6-7 [27, 31]

Partition 2 :- Left subarray [3, 9, 17, 5, 12]

Low = 0 high = 4 pivot = A[0] = 3

j = 1 compare A[j] < pivot $\rightarrow 9 < 3$ false } j++

no increment in i, only j increase, no swap

j = 2 compare A[j] < pivot $\rightarrow 17 < 3$ false } j++

no increment in i, only j increase, no swap

j = 3 compare A[j] < pivot $\rightarrow 5 < 3$ false } j++

no increment in i, only j increase, no swap

j = 4 compare A[j] < pivot $\rightarrow 12 < 3$ false } j++

no increment in i, only j increase no swap

Loop ends swap A[Low], A[i] $\rightarrow$ A[0], A[0] no change

current array : [3, 9, 17, 5, 12, 24, 27, 31] } pivot 3 already at index 0.

Left subarray : empty

Right subarray (1-4) = [9, 17, 5, 12]

Partition 3 : Subarray [9, 17, 5, 12]

Low = 1 high = 4 pivot = A[1] = 9

j = 2 compare A[j] < pivot $\rightarrow 17 < 9$ false } j++

no increment in i, only j increase, no swap

j = 3 compare A[j] < pivot $\rightarrow 5 < 9$ true } i++

~~i = 2~~ i = 2 swap A[i], A[j] $\rightarrow$ A[1], A[3]

array : [~~9, 17, 9, 12~~] [9, 5, 17, 12]

j = 4 compare A[j] < pivot $\rightarrow 12 < 9$ false } j++

no increment in i, only j increase, no swap

Loop ends swap A[Low], A[i] $\rightarrow$ A[1], A[2]

current array : [5, 9, 17, 12]

Array after Partition 3 :

[3, 5, 9, 17, 12, 24, 27, 31] } pivot lands at index 2

Left subarray (index 1) : [5] } base case

Right subarray (index 3-4) : [17, 12]

Partition 4: Subarray [17, 12]

low = 3 high = 4 pivot = $A[3] = 17$

$j = 4$ compare $A[j] < \text{pivot} \rightarrow 12 < 17$ true $\{ i++$

so $i = 4$, swap $A[i], A[j] \rightarrow A[4], A[4]$

Loop ends

swap $A[\text{low}], A[i] \rightarrow A[3], A[4]$

current array: [12, 17]

Array after Partition 4: [3, 5, 9, 12, 17, 24, 27, 31] $\left. \begin{array}{l} \text{pivot } 17 \\ \text{lands at} \\ \text{index } 4 \end{array} \right\}$

Left subarray: [index 3] = [12] $\{ \text{base case} \}$

Right subarray: empty

Partition 5: Subarray [27, 31]

low = 6 high = 7 pivot = $A[6] = 27$

$j = 7$ compare $A[j] < \text{pivot}$ $31 < 27$ false $\{ j++$

no i increase, no swap, only j increase

Loop ends

swap $A[\text{low}], A[i] \Rightarrow A[6], A[6]$

array unchanged pivot 27 already at index 6

Left subarray: empty

Right subarray: [31] base case

Final Sorted Array :- [3, 5, 9, 12, 17, 24, 27, 31]

Pivots used in order (24, 3, 9, 17, 27) all others were size-1 base case needing no partition.

Q3 iii) Loop invariant - Why Partition correctly places the pivot

Invariant:- At the start of each iteration of the for loop (for a given j) for all array indices k in the range (low... high):

- if $\text{low} + 1 \leq k \leq i$, then $A[k] < \text{pivot}$
- if $i + 1 \leq k \leq j - 1$, then $A[k] \geq \text{pivot}$

- $A[\text{low}]$ still holds the pivot value (it hasn't moved yet)
- elements $A[j..high]$ have not yet been examined.

Initialization :- Before the first iteration, $j = \text{low} + 1$ and $i = \text{low}$, so both ranges $A[\text{low} + 1..i]$ and $A[i + 1..j - 1]$ are empty, the invariant holds vacuously.

Maintenance : On each iteration, exactly two cases occur. If $A[j] \geq \text{pivot}$, we do nothing. [for $A[j]$ correctly joins the $\geq \text{pivot}$ region as j advances].
 • if $A[j] < \text{pivot}$. We increment i and swap $A[i]$ with $A[j]$. This moves the small element into the " $< \text{pivot}$ " region (extending it by 1) and pushes the previous boundary element into the " $\geq \text{pivot}$ " region.
 • Either way, both regions remain internally correct as j moves forward invariant preserved.

Termination: The loop ends when $j = \text{high} + 1$, meaning all elements $A[\text{low} + 1..high]$ have been classified:

$$A[\text{low} + 1..i] < \text{pivot}$$

$$A[i + 1..high] \geq \text{pivot}$$

The final swap ($A[\text{low}], A[i]$) moves the pivot out of position low and into position i , which is exactly the boundary between two regions. After this swap:

$$A[\text{low}..i + 1] < \text{pivot} \leq A[i] = \text{pivot} < A[i + 1..high]$$

This is exactly the partition property required. (the pivot sits in its final correct sorted position, with everything small to its left and everything not smaller to its right.)

[correctness proved].

Q4i) Balanced Team Division

Example data: Ratings $[10, 5, 7, 20, 12, 18, 9, 14]$

$$2n = 8 \text{ players}$$

$$\text{so } n = 4 \text{ players per team}$$

A simple $O(n^2)$ algorithm

Idea (greedy, no pre-sorting): Repeatedly find the highest rated remaining player and hand them to whichever team currently has the smaller total sum. This keeps "catching up" the weaker team so the totals stay close. Once a team already has n players, the rest automatically goes to the other team.

Algorithm:

BalanceTeams (ratings $[1..2n]$):

remaining = ratings

teamA = { }, teamB = { }

sumA = 0, sumB = 0

repeat $2n$ times:

x = max value in remaining // $O(n)$

remove x from remaining

if $|teamA| < n$ AND (sumA $\leq$ sumB OR $|teamB| == n$):

add x to teamA

sumA = sumA + x

else:

add x to teamB

sumB = sumB + x

why this costs $O(n^2)$:

Each of the $2n$ iteration does a linear scan ($O(n)$) over the shrinking "remaining" list to find the current maximum. Total cost = $2n * O(n) = O(n^2)$

Worked trace of given example rating

Pick	Value	sumA before	sumB before	Goes to	sumA	sumB
1	20	0	0	A (tie $\rightarrow$ A)	20	0
2	18	20	0	B (smaller)	20	18
3	15	20	18	B (smaller)	20	33
4	14	20	33	A (smaller)	34	33
5	12	34	33	B (smaller)	34	45
6	10	34	45	A (smaller)	44	45
7	9	44	45	A (smaller, now full)	53	45
8	7	—	—	force B (A full)	53	52

Result : Team A = {20, 14, 10, 4} sum = 53

Team B = {18, 15, 12, 7} sum = 52

Difference = 1 well balanced.

Q4ii) An $O(n \log n)$ approach using sorting

Strategy: The $O(n^2)$ cost above comes entirely from re-scanning for the maximum every round. If we sort all $2n$ ratings in "descending order" first, the (find current max) steps becomes $O(1)$. As we just take the next element in the sorted list. The rest of the greedy logic (assign to the team with the smaller running sum, respecting the capacity n limit) is unchanged.

Algorithm:-

Balanced Teams (ratings $[1..2n]$):

sort ratings in descending order // $O(n \log n)$.

sumA = 0, sumB = 0

countA = 0, countB = 0

for x in sorted ratings: // $O(n)$ total, $O(1)$ per element

if countA < n and (sumA <= sumB or countB == n):

add x to team A;

sumA += x ;

countA++;

else:

add x to team B;

sumB += x ;

countB++;

Running it on "20, 18, 15, 14, 12, 10, 9, 7" (already the descending order used in previous page) reproduces exactly the same trace and the same final team. The algorithm is identical in logic, just faster to execute.

Running Time analysis:

- Sorting: $O(n \log n)$ {merge sort or similar}.

- Single pass assigning each of the $2n$ elements: $O(1)$ work per element $\rightarrow O(n)$.

- Total: $O(n \log n) + O(n) = O(n \log n)$

Q5) Finding the optimal pairing by hand ($2n=6$, $n=3$ pairs)

Strategy: Pair the smallest with the largest, the second-smallest with the second largest and so on.

Sorted Ascending: 2, 4, 6, 8, 10, 12

Pair	Sum
(2, 12) →	14
(4, 10) →	14
(6, 8) →	14

All three pairs sum equal 14.

The maximum pair sum is 14. (This is best possible outcome as no pairing of these 6 numbers can make every pair sum below 14, since 12 alone must be paired with something and $12 + \text{anything} \geq 12 + 2 = 14$.)

5ii) $O(n \log n)$ algorithm.

Algorithm

Minimize Max Pair Sum ($A[1..2n]$):

sort A in ascending order; // $O(n \log n)$

$i=1$, $j=2n$

while $i < j$:

pair ($A[i]$, $A[j]$)

$i = i+1$

$j = j-1$

Running time: Sorting dominates at $O(n \log n)$. The pairing loop is single Pass, $O(n)$.

Total = $O(n \log n) + O(n) = O(n \log n)$. {ignore lower order factor, and constant factor}

Why optimal

Let the numbers be sorted: $a_1 \leq a_2 \leq \dots \leq a_{2n}$

The largest number a_{2n} must be paired with someone. To make its pair sum as small as possible, we should pair it with smallest number a_1 .

If a_{2n} is paired with a larger number instead. Replacing that partner with a_1 can only "decrease the largest pair sum". We then repeat the same idea for the remaining numbers. Therefore, pairing the smallest with the largest repeatedly minimizes the largest pair

sum. Hence, the strategy is optimal.

Q6a) $n^2 + 5n + 10 = O(n^2)$

need to find c and n where $c, n > 0$.

$$n^2 + 5n + 10 \leq cn^2$$

Method :- Bound each term individually, then add

$$n^2 \leq n^2 \quad \left\{ \begin{array}{l} \text{true for all } n \geq 0 \end{array} \right.$$

$$5n \leq 5n^2 \quad \left\{ \begin{array}{l} \text{true whenever } n \geq 1, \text{ works for } n=0 \text{ as well} \end{array} \right.$$

$$10 \leq 10n^2 \quad \left\{ \begin{array}{l} \text{true whenever } n \geq 1 \end{array} \right.$$

Adding these three inequalities (valid together once $n_0 \geq 1$)

$$n^2 + 5n + 10 \leq n^2 + 5n^2 + 10n^2$$

$$n^2 + 5n + 10 \leq 16n^2 \quad \text{for all } n \geq 1$$

Conclusion :- $n^2 + 5n + 10 = O(n^2)$ for $c=16, n_0=1$

Q6b) $4n^2 \log_2 n + 3n^2 = O(n^2 \log_2 n)$

need to find c and n where $c, n > 0$

$$4n^2 \log_2 n + 3n^2 \leq cn^2 \log_2 n$$

Method :- Bound each term against $n^2 \log_2 n$

$$\bullet 4n^2 \log_2 n \leq 4n^2 \log_2 n \quad (\text{trivially true always})$$

$$\bullet 3n^2 \leq 3n^2 \log_2 n$$

↳ solve

$$\frac{3n^2}{3n^2} \leq \log_2 n$$

$$1 \leq \log_2 n$$

$$2^1 \leq n$$

so we conclude that

$$3n^2 \leq 3n^2 \log_2 n$$

when

$$\log_2 n \geq 1 \quad \text{that is } n \geq 2$$

Adding the two bounds

$$4n^2 \log_2 n + 3n^2 \leq 4n^2 \log_2 n + 3n^2 \log_2 n$$

$$4n^2 \log_2 n + 3n^2 \leq 7n^2 \log_2 n \quad \text{for all } n \geq 2$$

Data

Big O so tight upper bound

$$f(n) = O(g(n))$$

$$f(n) \leq c \cdot g(n)$$

$$f(n) = n^2 + 5n + 10$$

$$g(n) = n^2$$

Data

Big O so tight upper bound

$$f(n) = O(g(n))$$

$$f(n) \leq c \cdot g(n)$$

$$c, n_0 > 0$$

$$f(n) = 4n^2 \log_2 n + 3n^2$$

$$g(n) = n^2 \log_2 n$$

Conclusion: $4n^2 \log_2 n + 3n^2 = O(n^2 \log_2 n)$ for $c = 7$. $n_0 = 2$

Q7i) The purpose is to describe how an algorithm's running time scales as the input size n grows, without getting distracted by exact operation counts, hardware speed, or programming-language overhead. Instead of measuring "this takes 2.5sec on my laptop", asymptotic notation lets us say "this takes on the order of n^2 step".

A statement that stays true regardless of which machine or language runs it. It focuses attention on the algorithm's inherent scalability: what happens when the data set becomes very large, since that's when the difference b/w a good and a bad algorithm actually shows up.

ii) All three compare a function $f(n)$ (the real running time) against a simpler reference function $g(n)$, using a constant multiplier " c " and a threshold point n_0 , beyond which the comparison must hold:

- Big O : It gives an upper bound. $f(n)$ never grows faster than $c \cdot g(n)$ once $n \geq n_0$. It answers "how bad can this get at worst?"
- Big Ω : It gives a lower bound. $f(n)$ never grows slower than $c \cdot g(n)$ once $n \geq n_0$. It answers "how good can this possibly be, at best?"
- Big Θ : It combines both. It requires two constants c_1 and c_2 , such that $f(n)$ is sandwiched between $c_1 \cdot g(n)$ and $c_2 \cdot g(n)$ forever after n_0 . This means $g(n)$ isn't just some safe ceiling or floor. It's genuinely the same growth rate as $f(n)$, in both directions.

So Big O is a "no worse than" statement

Big Ω is a "no better than" statement and,

Big Θ is "exactly this rate". The tightest and most informative of the three.

iii) Constants and lower order terms depend on things that have nothing to do with the algorithm's actual logic (clock speed; compiler organizations; how operations are counted).

They also become irrelevant once n is large enough. (eg $+100n$ is replaced by n^2 once n passes a few hundred, no matter how big that 100 is). Since asymptotic notation is only concerned with behavior as $n \rightarrow \infty$ (large inputs), the dominant term is the only one that matters in the long run. Hence why constant and lower order are ignored.

iv) Linear Search:- Scan every element one by one until found. Runs in $O(n)$ time. In the worst case it checks every element.

Binary Search:- Repeatedly halve the search range in a sorted array. Runs in $O(\log n)$ time.

eg:- For a list of a million elements, linear search might need up to a million comparisons. In the worst case, while binary search needs only about 20.

A huge practically significant difference that becomes obvious when seen w.r.t growth.

v) Summary

The lecture builds the 3 bound geometrically before giving formal definitions. It starts with real valued functions $f(x)$ and $g(x)$.

If some constant c can be found so that

- $c \cdot g(x)$ stays above $f(x)$ from some point x_0 onward. f is in $O(g)$
- $c \cdot g(x)$ stays below $f(x)$ from some point x_0 onward. f is in $\Omega(g)$.

and if two constants c_1, c_2 sandwich $f(x)$ from both sides at once f is in $\Theta(g)$. It then repeats the exact same idea for functions defined only on the positive integers, rather than the real line.

Two important concepts:-

- The graphical picture and the formal definition are two views of the same idea. Visualising functions being trapped under, over etc makes the symbolic definition ($\exists c, \forall c$, such that) far less abstract.
- The choice of n_0 exists specifically to let us ignore small irregular behavior. Hence why asymptotic analysis ignores whatever an algorithm does on tiny inputs and focus

purely on large scale trends.

Q8i) $T(n) = \sqrt{2} T(\frac{n}{2}) + \log n$

$a = \sqrt{2}$ $b = 2$ $d = 1$

$a \leq b^d \Rightarrow \sqrt{2} \leq 2^1 = 2$

$\therefore$ case 2

$T(n)$

can't define d as it isn't pure polynomial

8i) $T(n) = \sqrt{2} T(\frac{n}{2}) + \log n$

$a = \sqrt{2}$, $b = 2$, $f(n) = \log(n)$

calc: $n^{\log_b a}$

$= n^{\log_2 \sqrt{2}}$

$= n^{0.5}$

$= \sqrt{n}$

Compare $f(n)$ against $n^{0.5}$

so $f(n) < n^{0.5}$ case 1

$\therefore T(n) = \Theta(n^{\log_b a})$

$T(n) = \Theta(n^{\log_2 \sqrt{2}})$

$= \Theta(n^{0.5 \log_2 2})$

$= \Theta(n^{0.5})$

$T(n) = \Theta(\sqrt{n})$

Answer

8ii) $T(n) = 64 T(\frac{n}{8}) - n^2 \log n$

$a = 64$ $b = 8$

$f(n) = -n^2 \log n$

as $f(n)$ is not positive it does not solve

calc: $n^{\log_b a}$

$n^{\log_8 64}$

$\Rightarrow n^{2 \log_8 8}$

$\Rightarrow n^2$

Compare $f(n)$ against n^2

so $f(n) = n^2$

$2^3 \log 2 = 2^2$
 $4 = 4$

let $k=1$

$\therefore T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$

$T(n) = \Theta(n^{\log_8 64} \log^{1+1} n)$

$T(n) = \Theta(n^2 \log^2 n)$

Answer

Master Theorem case

Case 1: $a = b^d$

$O(n^d \log_b n)$

Case 2: $a < b^d$

$O(n^d)$

Case 3: $a > b^d$

$O(n^{\log_b a})$

case 1

$f(n) < n^{\log_b a}$

then $T(n) = \Theta(n^{\log_b a})$

case 2

$f(n) = n^{\log_b a}$

then $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$

case 3

$f(n) > n^{\log_b a}$

then $T(n) = \Theta(f(n))$

8iii) $T(n) = 16T\left(\frac{n}{4}\right) + n!$

$a = 16$ $b = 4$ $f(n) = n!$

calc: $n^{\log_b a}$

$\Rightarrow n^{\log_4 16} \Rightarrow n^{2 \log_4 4} \Rightarrow n^2$

Compare $f(n)$ against n^2

$f(n) > n^2$ } factorial growth is vastly larger than any polynomial

Check regularity condition for case 3

$a \cdot f\left(\frac{n}{b}\right) \leq c \cdot f(n)$ for some constant $c < 1$ and large n

eg $16 \cdot \left(\frac{n}{4}\right)! \leq c \cdot n!$

since factorial shrinks super exponentially when its argument is divided by 4. Therefore it holds easily for large n

hence $T(n) = O(f(n))$

$T(n) = O(n!)$ answer.

Q9i) $T(n) = 2T\left(\frac{n}{2}\right) + n \log n$

Step 1: expand

substitute $T\left(\frac{n}{2}\right)$: first calc $T\left(\frac{n}{2}\right)$

$T\left(\frac{n}{2}\right) = 2T\left(\frac{n}{4}\right) + \frac{n}{2} \log \frac{n}{2}$

$T(n) = 2\left(2T\left(\frac{n}{4}\right) + \frac{n}{2} \log \frac{n}{2}\right) + n \log n$

$T(n) = 4T\left(\frac{n}{4}\right) + n \log \frac{n}{2} + n \log n$

expand again:

$T\left(\frac{n}{4}\right) = 2T\left(\frac{n}{8}\right) + \frac{n}{4} \log \frac{n}{4}$

substitute

$T(n) = 4\left(2T\left(\frac{n}{8}\right) + \frac{n}{4} \log \frac{n}{4}\right) + n \log \frac{n}{2} + n \log n$

$= 8T\left(\frac{n}{8}\right) + n \log \frac{n}{4} + n \log \frac{n}{2} + n \log n$

expand again

$T\left(\frac{n}{8}\right) = 2T\left(\frac{n}{16}\right) + \frac{n}{8} \log \frac{n}{8}$

substitute: $T(n) = 8 \left(2T\left(\frac{n}{16}\right) + \frac{n}{8} \log \frac{n}{8} \right) + n \log \frac{n}{4} + n \log \frac{n}{2} + n \log n$

$$= 16T\left(\frac{n}{16}\right) + n \log \frac{n}{8} + n \log \frac{n}{4} + n \log \frac{n}{2} + n \log n$$

Step 2: Identify pattern
after k levels

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + n \sum_{i=0}^{k-1} \log\left(\frac{n}{2^i}\right)$$

Step 3: Stopping condition

We stop when the subproblem becomes 1

$$\frac{n}{2^k} = 1$$

$$2^k = n$$

$$k = \log_2 n$$

Step 4: Calculate Total Work

at each level the work is approximately $n \log n$
but logarithm decreases by 1 at each level

$$n [\log n + (\log n - 1) + (\log n - 2) + \dots + 1]$$

so

$$T(n) = O(n \log^2 n)$$

Ans

The bracket is sum and

$\log n - 1$
 $\log n - 2$
the come from
 $\log \frac{n}{2} = \log n - \log 2$
 $\Rightarrow \log n - 1$
and such

$1+2+\dots+m$
 $= O(m^2)$ so $1+2+\dots+\log n$ is
 $O(\log^2 n)$

Q9ii) $T(n) = 8T\left(\frac{n}{2}\right) + n^2$

calc

$$T\left(\frac{n}{2}\right) = 8T\left(\frac{n}{4}\right) + \frac{n^2}{4}$$

$$T\left(\frac{n}{4}\right) = 8T\left(\frac{n}{8}\right) + \frac{n^2}{16}$$

$$T\left(\frac{n}{8}\right) = 8T\left(\frac{n}{16}\right) + \frac{n^2}{64}$$

substitute

$$T(n) = 8 \left(8 \left(8 \left(8T\left(\frac{n}{16}\right) + \frac{n^2}{64} \right) + \frac{n^2}{16} \right) + \frac{n^2}{4} \right) + n^2$$

$$= 8 \left(8 \left(8 \left(8T\left(\frac{n}{16}\right) + \frac{n^2}{64} \right) + \frac{n^2}{16} \right) + \frac{n^2}{4} \right) + n^2$$

$$= 8 \left(8 \left(64T\left(\frac{n}{16}\right) + \frac{n^2}{8} + \frac{n^2}{16} \right) + \frac{n^2}{4} \right) + n^2$$

$$= 8 \left(512T\left(\frac{n}{16}\right) + n^2 + \frac{n^2}{2} + \frac{n^2}{4} \right) + n^2$$

$$= 4096T\left(\frac{n}{16}\right) + 8n^2 + 4n^2 + 2n^2 + n^2$$

Step 2: Identify pattern
after k levels.

$$T(n) = 8^k T\left(\frac{n}{2^k}\right) + n^2(1+2+4+8+\dots+2^{k-1})$$

the geometric sum is

$$1+2+4+8+\dots+2^{k-1} = 2^k - 1$$

So $T(n) = 8^k T\left(\frac{n}{2^k}\right) + n^2(2^k - 1)$

Step 3: Stopping condition.

stop when $\frac{n}{2^k} = 1$

$$n = 2^k$$

$$k = \log_2 n$$

substitute

since $8 = 2^3$

last term is $n^2(2^k)$ as $n = 2^k$
 $n^2(n)$
 n^3

Calculate total work

$$\begin{aligned} \sum_{i=0}^{k-1} n^2 \cdot 2^i &= n^2(2^k - 1) \\ &= n^2(n - 1) \\ &= \Theta(n^3) \end{aligned}$$

$$\therefore T(n) = \Theta(n^3)$$

Q10a) True. By definition $O(f(n))$ means some function grows at most as fast as $f(n)$. Hence $f(n) + O(f(n))$ is still bounded above and below by constant multiples of $f(n)$

eg: $f(n) = n^2$

$$n^2 + O(n^2) = \Theta(n^2) \text{ proof}$$

b) True :- This follows from the transitive property of Big- O
 that if $f(n) \leq c_1 g(n)$ and $g(n) \leq c_2 h(n)$
 then $f(n) \leq c_1 c_2 h(n)$
 $\therefore f(n) = O(h(n))$

~~True: $f(n) = O(g(n))$ means $\exists c_1, c_2, n_0$ with $c_1 g(n) \leq f(n) \leq c_2 g(n)$ for $n > n_0$~~

c) True, Since $f(n) = O(g(n))$ $f(n)$ and $g(n)$ have same growth rate

So eg: $f(n) = 3n^2$, $g(n) = n^2$

$$f(n) + g(n) = 3n^2 + n^2 = 4n^2$$

and $4n^2 = O(n^2)$

$\therefore f(n) + g(n) = O(g(n))$ is proved

d) True :- for $n \geq 1$: $n^2 \leq n^3 \therefore$ using $c=1$

the $n^2 \leq 1 \cdot n^3$ is valid

Hence $n^2 = O(n^3)$

e) False: For Big O, n^3 would need to be bounded by a constant multiple of n^2 :

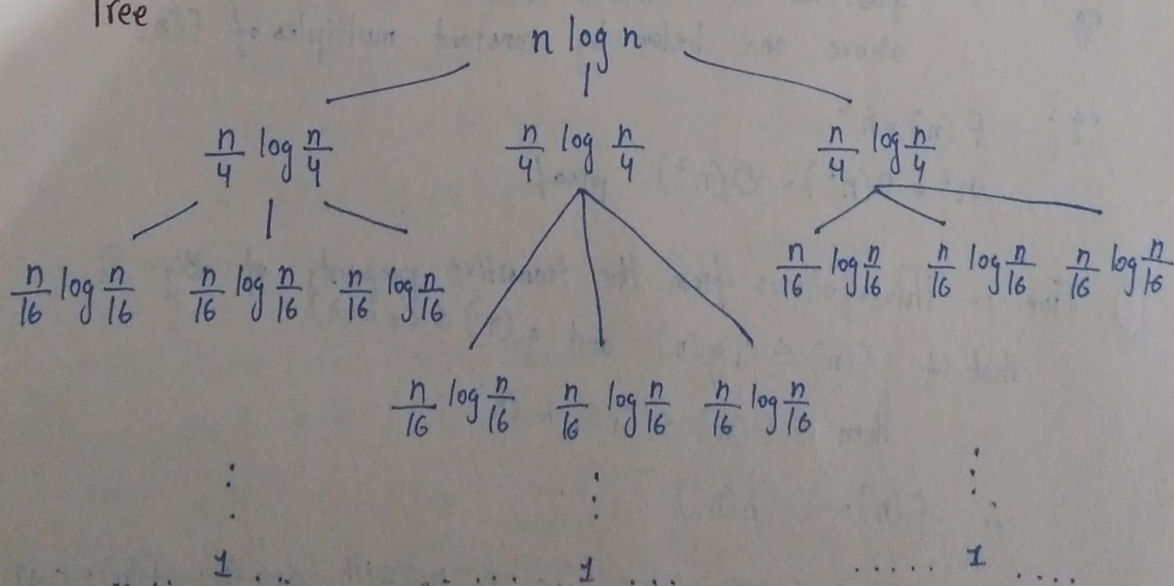
$$n^3 \leq cn^2$$

$$n \leq c$$

But n keeps increasing and can't be bounded by a fixed constant c . Hence $n^3 \neq O(n^2)$

Q11a) $T(n) = 3T\left(\frac{n}{4}\right) + n \log n$, $T(1) = O(1)$

Tree



Step 2:- cost at level i

$$\text{No. of nodes} = 3^i$$

$$\text{Size of each subproblem} = \frac{n}{4^i}$$

$$\text{Work at each node} = \frac{n}{4^i} \log\left(\frac{n}{4^i}\right)$$

$$\begin{aligned}\therefore \text{Total cost} &: 3^i \left[\frac{n}{4^i} \log \frac{n}{4^i} \right] \\ &= n \left(\frac{3}{4} \right)^i \log \left(\frac{n}{4^i} \right)\end{aligned}$$

Step 3: Tree Height

Stop when the subproblem size becomes 1. (h = height)

$$\frac{n}{4^h} = 1$$

$$n = 4^h$$

$$h = \log_4 n \quad \text{Answer}$$

Step 4: Find Dominant Level.

As " i " increases both factor shrinks. The geometric factor $\left(\frac{3}{4}\right)^i$ it decreases that the ratio < 1 , and $\log\left(\frac{n}{4^i}\right)$ also decreases. So every term in the level cost sequence is smaller than the one before it. The root (level 0) dominates.

$$\text{Root cost} = n \log n$$

$$\therefore T(n) = \Theta(n \log n)$$

$$\Theta(n \log n)$$

} asymptotic bound

cross check via Master theorem

$$a=3 \quad b=4 \quad f(n) = n \log n$$

$$\text{calc } n^{\log_b a} = n^{\log_4 3} = n^{0.7942}$$

compare $f(n)$ with $n^{0.7942}$

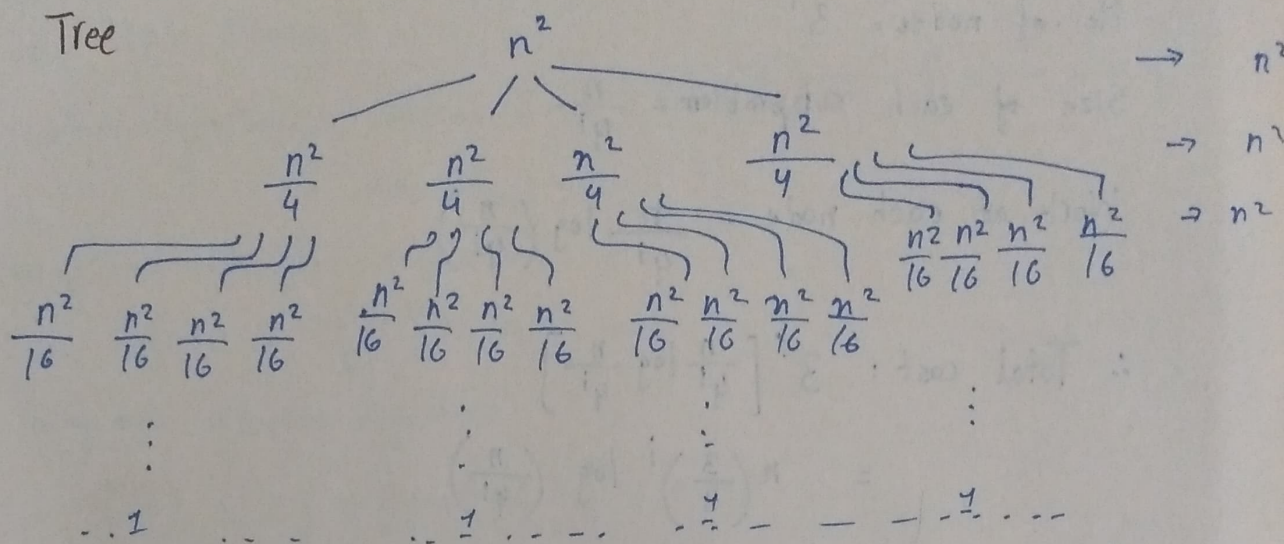
$$f(n) > n^{0.7942} \quad \text{when } n=2 \quad 2 > 1.73$$

case 3 so

$$\Theta(f(n)) = \Theta(n \log n) \quad \text{proof}$$

Q11b) $T(n) = 4T\left(\frac{n}{2}\right) + n^2$, $T(1) = \theta(1)$

Tree



Every Level has n^2 work

Level 0 = n^2

Level 1 = $4 \frac{n^2}{4} = n^2$

Level 2 = $16 \frac{n^2}{16} = n^2$

} cost at each level

$\Rightarrow$ Number of Level:

stop when $\frac{n}{2^h} = 1$

$2^h = n$

$h = \log_2 n$

$\Rightarrow$ Total cost:-

There are approximately $\log_2 n$ levels and each level costs n^2

$\therefore T(n) = n^2 + n^2 + n^2 + \dots$ for $\log n$ levels

$\therefore T(n) = n^2 \log n$

so $T(n) = \theta(n^2 \log n)$

There is no single internal level dominate because every internal level has exactly the same cost n^2 . No level is larger than the others. Therefore we have to add the cost of all $\log n$ levels.

$n^2 \times \log n = \theta(n^2 \log n)$

Q12a) $T(n) = 2T\left(\frac{n}{2}\right) + n \log n$

Guess: $T(n) = O(n \log^2 n)$

Induction Hypothesis: - Assume $T\left(\frac{n}{2}\right) \leq c\left(\frac{n}{2}\right) \log^2\left(\frac{n}{2}\right)$ for some constant $c > 0$.

Substitute into the recurrence:

$$T(n) = 2T\left(\frac{n}{2}\right) + n \log n$$

using our induction hypothesis

$$T(n) \leq 2 \left[c \frac{n}{2} \log^2\left(\frac{n}{2}\right) \right] + n \log n$$

$$T(n) \leq cn \log^2\left(\frac{n}{2}\right) + n \log n$$

using $\log\left(\frac{n}{2}\right) = \log n - \log_2 2 = \log n - 1$

$$T(n) \leq cn (\log n - 1)^2 + n \log n$$

expand

$$T(n) \leq cn (\log^2 n - 2 \log n + 1) + n \log n$$

$$T(n) \leq cn \log^2 n - 2cn \log n + cn + n \log n$$

Grp the terms

$$T(n) \leq cn \log^2 n + n \log n (1 - 2c) + cn$$

we want $T(n) \leq cn \log^2 n$

This will happen when

$$n \log n (1 - 2c) + cn \leq 0$$

choose $c = 1$

$$T(n) \leq n \log^2 n + n \log n (1 - 2) + n$$

$$T(n) \leq n \log^2 n - n \log n + n$$

$$T(n) \leq n \log^2 n + \underbrace{n(1 - \log n)}_{\text{this is } \leq 0 \text{ for all } n \geq 2}$$

so $T(n) \leq n \log^2 n = cn \log^2 n$

Base case: $n_0 = 2 \Rightarrow T(2) = 2 \log^2 2 = O(1)$ a fixed constant

Choose c large enough so that $c 2 \log^2 2 = 2c \geq T(2)$

Conclusion $T(n) = O(n \log^2 n)$

$$12b) T(n) = 2T\left(\frac{n}{4}\right) + \sqrt{n}, \quad T(1) = O(1)$$

Guess 1 = $T(n) = O(\sqrt{n})$ fails why cause

$$T(n) \leq c\sqrt{2}$$

Inductive hypothesis

$$T\left(\frac{n}{4}\right) \leq c\sqrt{\frac{n}{4}} \rightarrow c\frac{\sqrt{n}}{2}$$

Inductive Hypothesis:

$$T\left(\frac{n}{4}\right) \leq c\sqrt{\frac{n}{4}} \log \frac{n}{4}$$

for some constant $c > 0$

$$T(n) \leq \frac{2c\sqrt{n}}{2} + \sqrt{n} = c\sqrt{n} + \sqrt{n} = (c+1)\sqrt{n}$$

this requires

$$(c+1)\sqrt{n} \leq c\sqrt{2} \quad \text{i.e. } 1 \leq 0$$

impossible for any c . so the guess was wrong

Substitute:

$$T(n) = 2T\left(\frac{n}{4}\right) + \sqrt{n}$$

using our inductive hypothesis

$$T(n) \leq 2 \left[c\sqrt{\frac{n}{4}} \log \frac{n}{4} \right] + \sqrt{n}$$

$$T(n) \leq c\sqrt{n} \log \frac{n}{4} + \sqrt{n}$$

$$\text{since } \log \frac{n}{4} = \log n - \log 4 = \log n - \log 2^2 = \log n - 2$$

$$T(n) \leq c\sqrt{n} (\log n - 2) + \sqrt{n}$$

$$T(n) \leq c\sqrt{n} \log n - 2c\sqrt{n} + \sqrt{n}$$

$$T(n) \leq c\sqrt{n} \log n + \sqrt{n} (1 - 2c)$$

$$\text{choose } c \geq \frac{1}{2}$$

$$\text{Then } 1 - 2c \leq 0$$

$$\therefore T(n) \leq c\sqrt{n} \log n \quad \text{Hence } T(n) = O(\sqrt{n} \log n) \text{ done}$$

The logarithmic factor necessary because this recurrence sits exactly on the Master Theorem case 2 boundary, here $a=2$, $b=4$, $d=\frac{1}{2}$ and $b^d = 2 = a$ i.e. $a=b^d$ exactly. ~~Whenever~~ Every level of the recursion tree contributes equal cost, and summing equal cost over $\log_4 n$ levels produce an extra multiplicative $\log n$ factor. Without this log factor, the induction leftover term can be made ≤ 0